

A Lease-Gated Architecture for Safe and Modular Autonomous Drone Systems

Brendan Waldrop
Barett, The Honors College and
School of Computing and Augmented Intelligence
Spring 2026

Thesis Director: Adwith Malpe
Secondary Committee Member: Dr. Ryan Meuth

Table of Contents

1. Introduction
2. Background Research
3. Existing Solutions
4. Architecture
5. Developer Workflow
6. Limitations and Challenges

Project Goals

- Describe an open-source companion computer architecture focused on:
 - Deterministic Safety
 - Modular Architecture
- Help bridge the gap between tool documentation and production.
- Provide a reproducible development workflow.



PX4 Autopilot Logo



ROS2 Framework Logo

Drone as a Solution

- Agriculture
- Public Safety
- Surveying
- Logistics

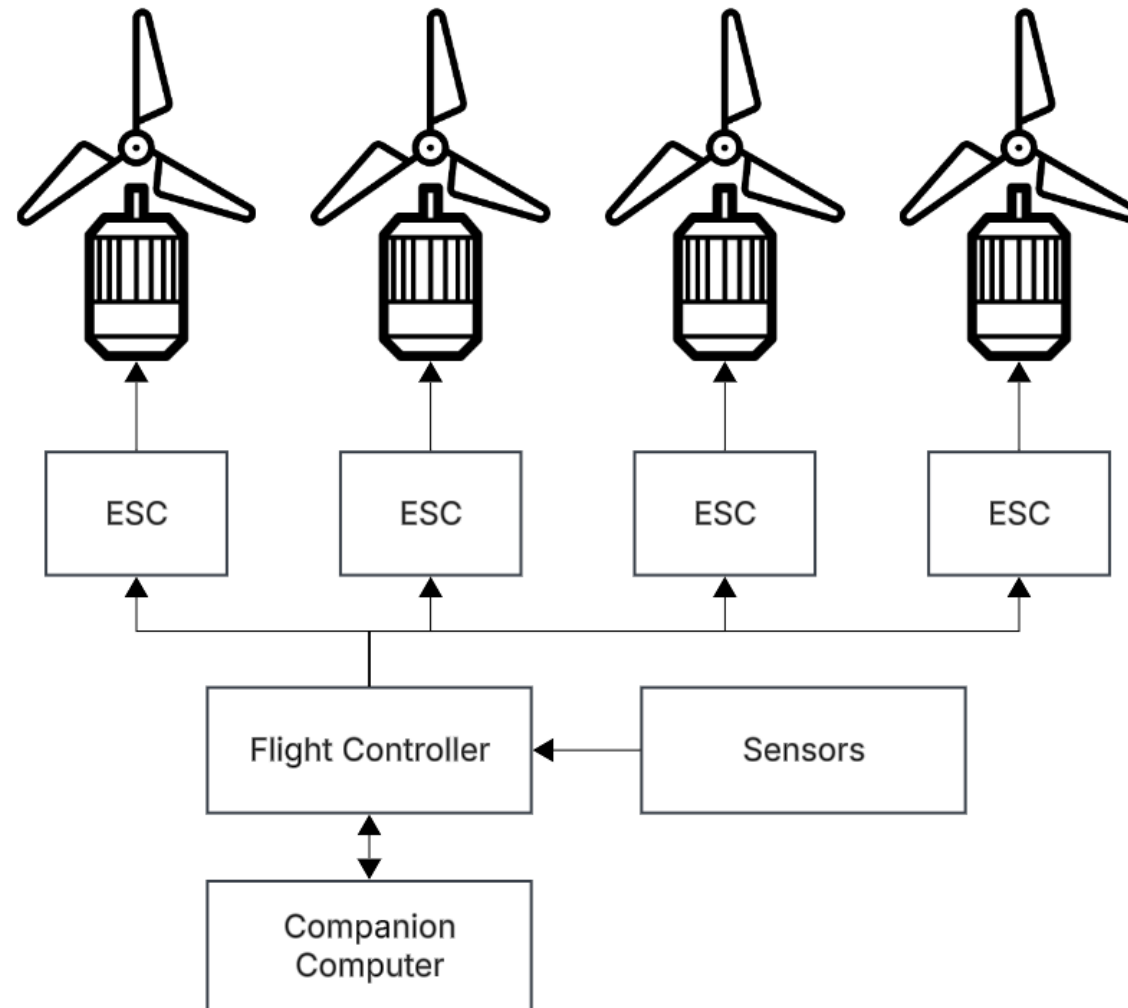


Drone in Agriculture Sector



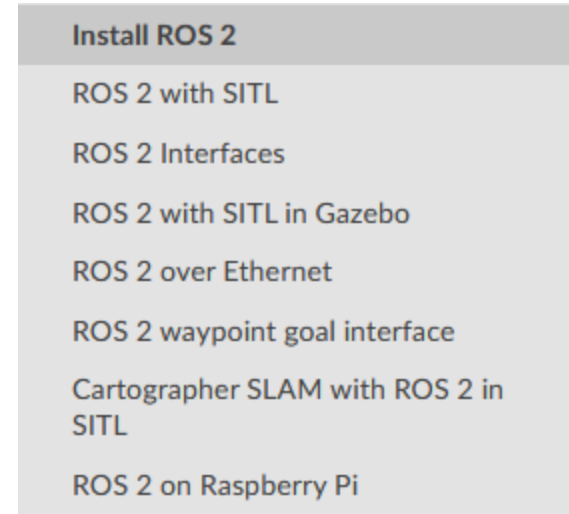
Amazon Delivery Drone

How Does a Drone Work?

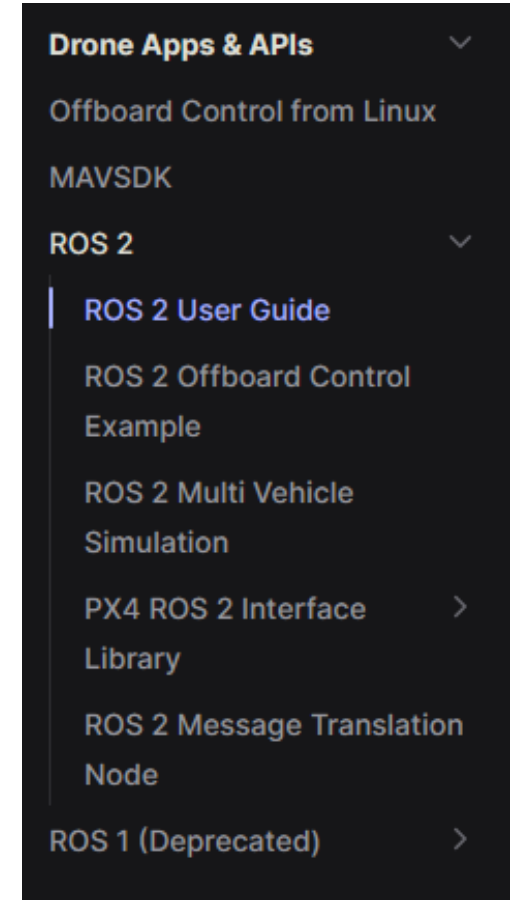


What is the Development Gap?

- Current open-source resources:
 - Only provide enough to get started.
 - Have minimal examples.
- Everything between there and deployment is up to the developer.



ArduCopter ROS2 Guide



PX4 ROS2 Guide

What is the Development Gap?

- Questions developers must ask:
 - What are "Best Practices"?
 - How do I do ...?
 - What's the best way to manage ...?
- The answers to these questions are:
 - Proprietary company information.
 - Strongly coupled with internal tech.
 - Limited with current open-source knowledge.



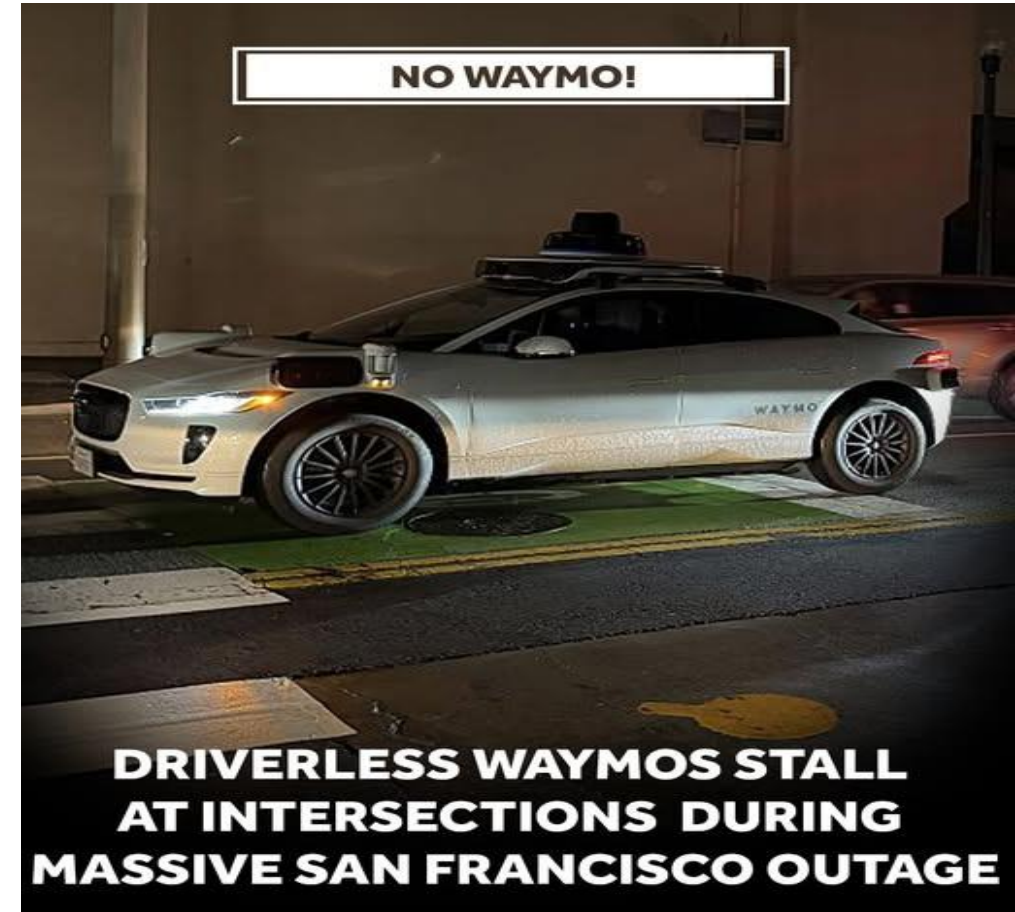
Skydio X10 on Launch Pad



*Anduril's US Airforce
"Ghost Drone"*

Ground vs. Aerial Robotics

- What's more consequential?
 - (A) A Waymo stopping in the middle of a road.
 - (B) A Drone freefalling from the sky.
- A default state in ground robotics:
 - Idle in place on the ground.
- A default state in aerial robotics:
 - Idle in place in the sky.
- Handling safety should always be a first-class priority.



Background Research

PX4 Autopilot

- Open-source autopilot software. [1]
- Widely used in research and commercial systems.
- Operates in discrete flight modes:
 - Takeoff
 - Land
 - Hold
 - Offboard
- Offboard flight modes allows an external system to send commands.



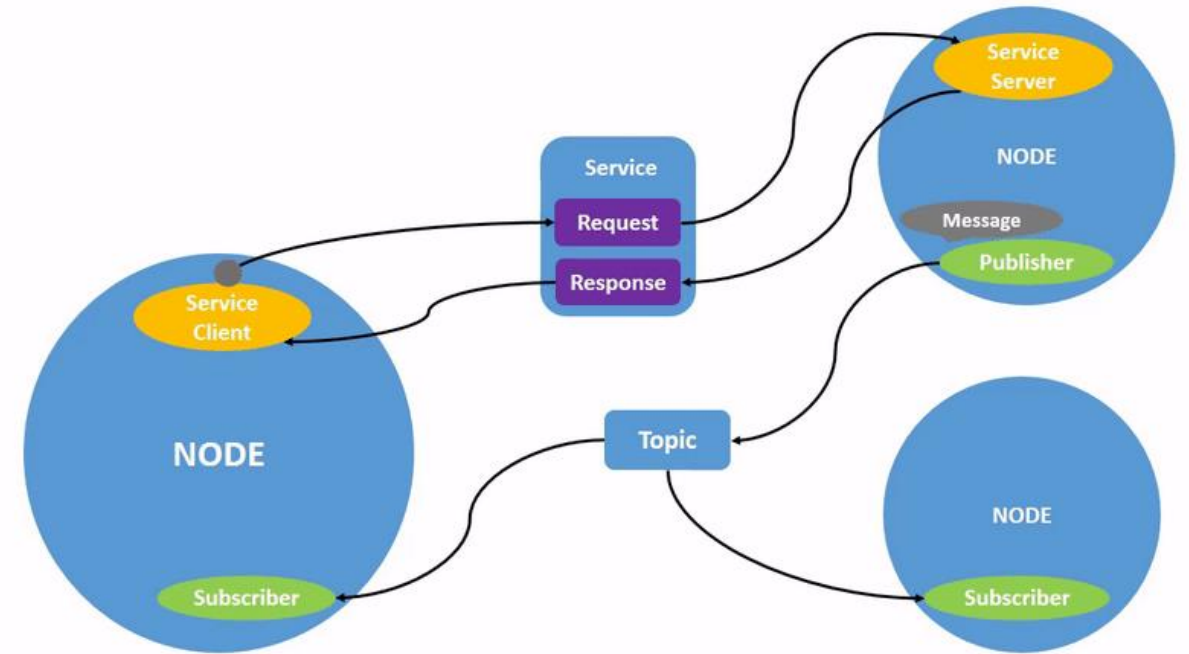
PX4 Autopilot Logo



Pixhawk V6X Autopilot Hardware

ROS2

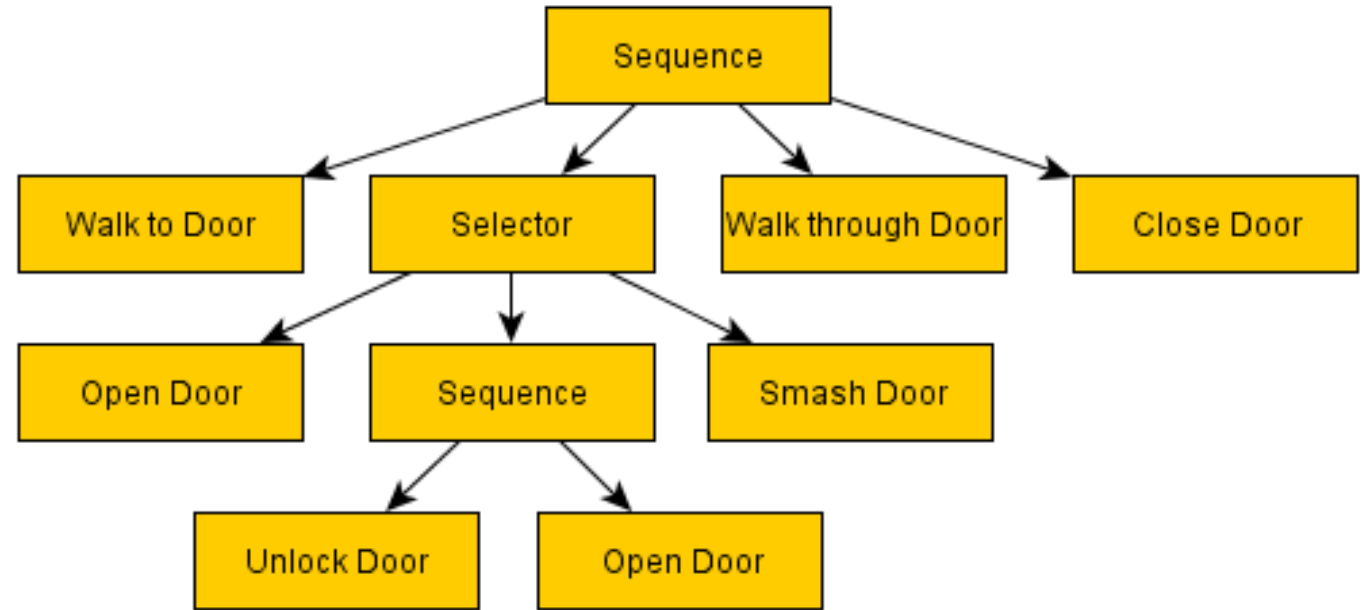
- Robot Operating System [2]
- Open-source middleware framework
- Distributed publisher-subscriber communication model.
- Nodes (Processes) communicate through topics.
- Support for managing node startup, monitoring, and teardown.



ROS2 Node Communication Graph

Behavior Tree

- Data structure for building decision-making logic. [7]
- Describes a set of tasks to complete in a modular fashion.
- Directed tree, nodes are classified into control nodes or executions nodes.



Behavior Tree Example

Existing Solutions

Aerostack2

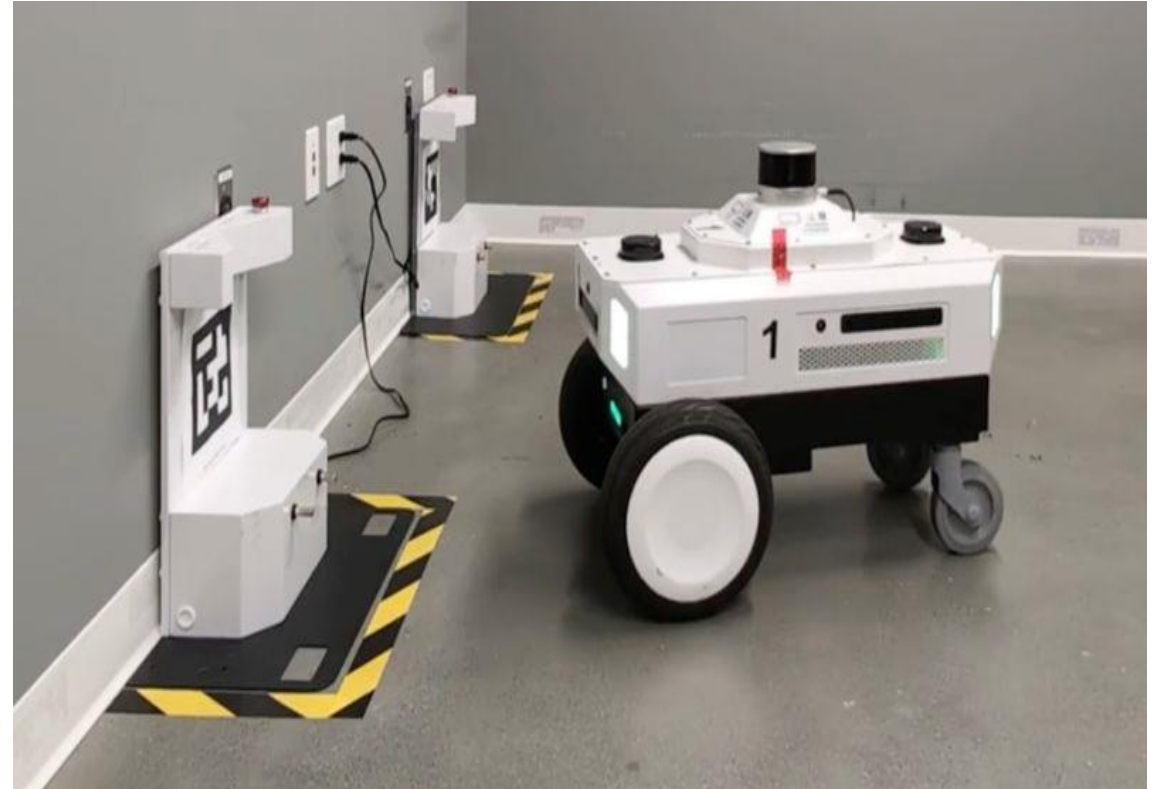
- Open-source framework for companion computers built on ROS2. [3]
- Pros:
 - Platform independence.
 - Behavior tree driven mission control.
 - Low barrier of entry.
- Cons:
 - Safety enforcement is not a first-class design concern.
 - Cannot "pick and choose" features for your use-case. Tightly coupled.



Aerostack2 Logo

Nav2

- Autonomous navigation framework for companion computers. [4]
- Pros:
 - Widely adopted, so design patterns are well validated.
 - Behavior tree mission management.
- Cons:
 - Specifically designed for ground robotics.
 - Design assumptions are rooted in the physics of ground vehicles.



Ground Robot running Nav2

MAVSDK

- Libraries for creating software that interacts with MAVLink-based vehicles. [6]
- Pros:
 - Exposes MAVLink services to application code.
 - Covers common flight operations including telemetry and mission execution.
- Cons:
 - API for communication, not a framework.

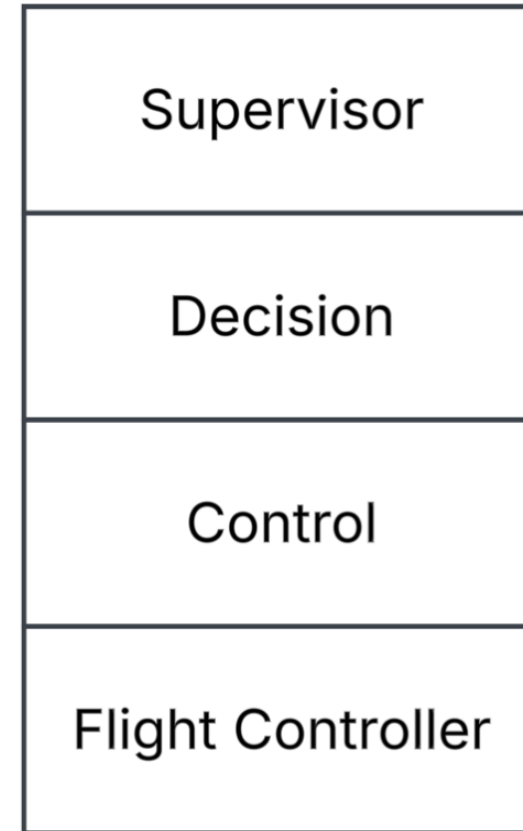


MAVSDK Logo

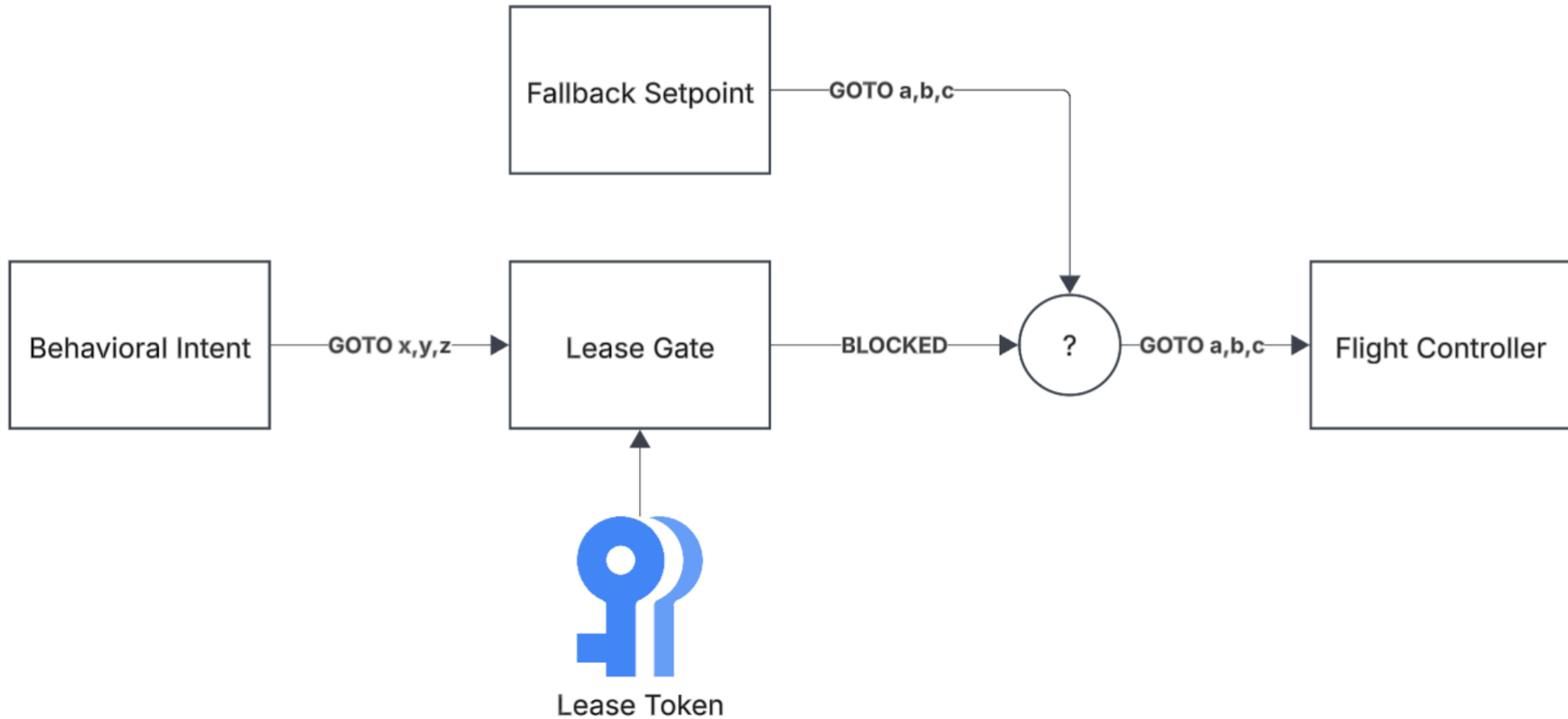
Architecture

UAV Lease-Gated Autonomy Stack

- Autonomy should always be explicitly authorized and never an assumed state.
- Composed of 3 Layers:
 - Supervisor Layer
 - Control Layer
 - Decision Layer



What is a Lease Gate?

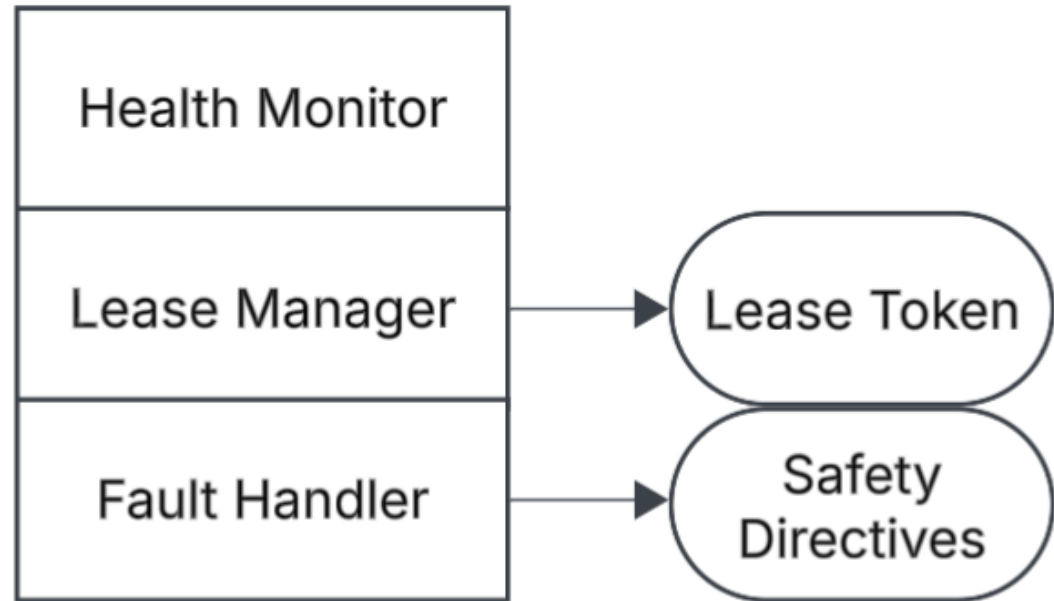


Properties of a Lease Gate

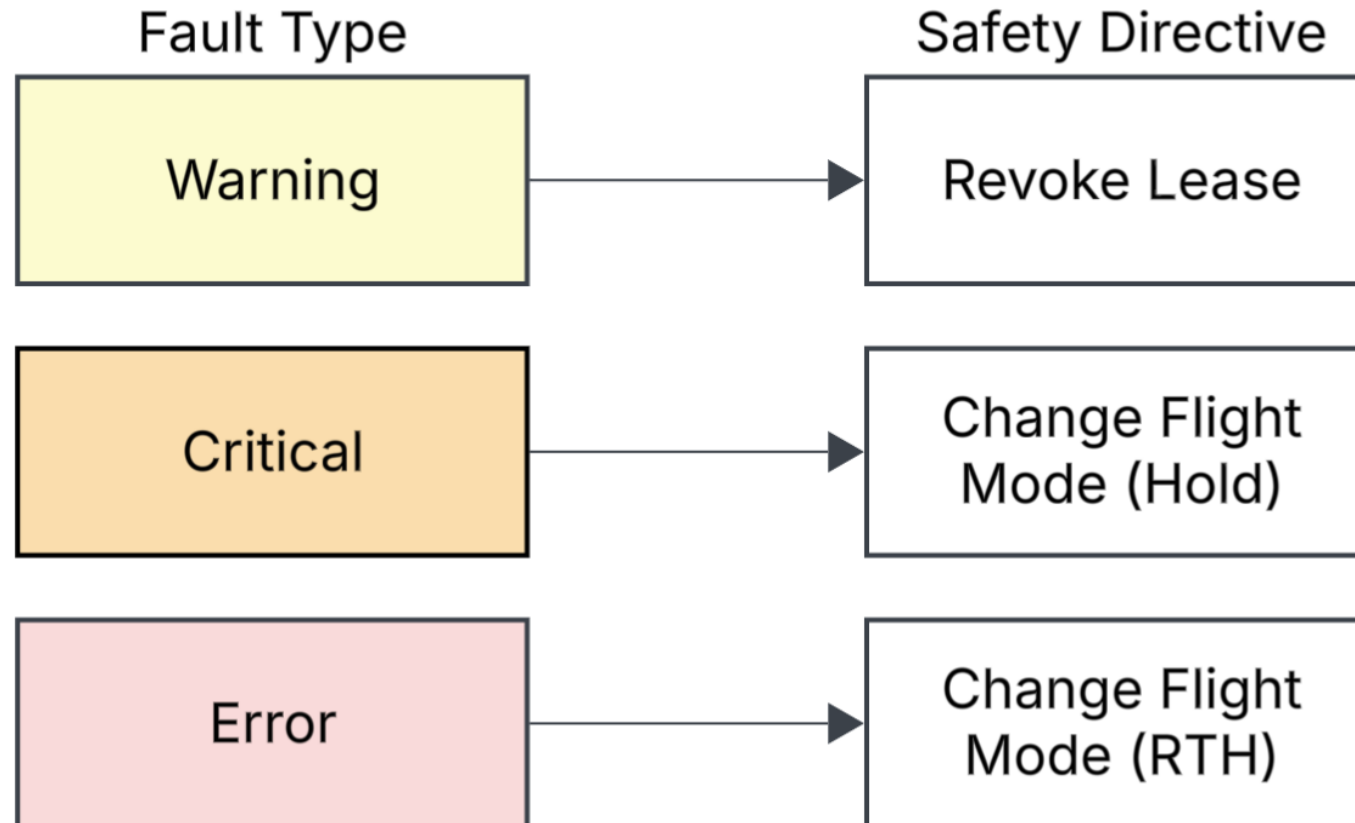
- Exists in two states:
 - Granted
 - Revoked/Expired
- Short-lived token
 - Continuously renewed by supervisor.
- When is it issued?
 - When safety conditions are satisfied.

Supervisor Layer

- Ultimate authority for autonomous operation.
- Responsibilities are divided across three components:
 - Lease Manager
 - Health Monitor
 - Fault Handler

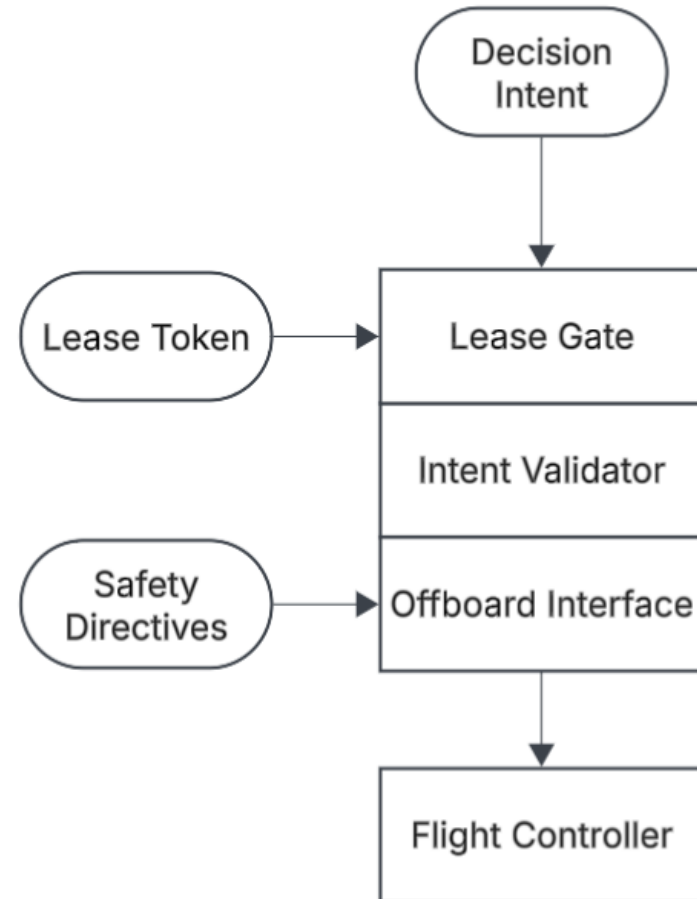


Supervisor Layer – Fault Handling



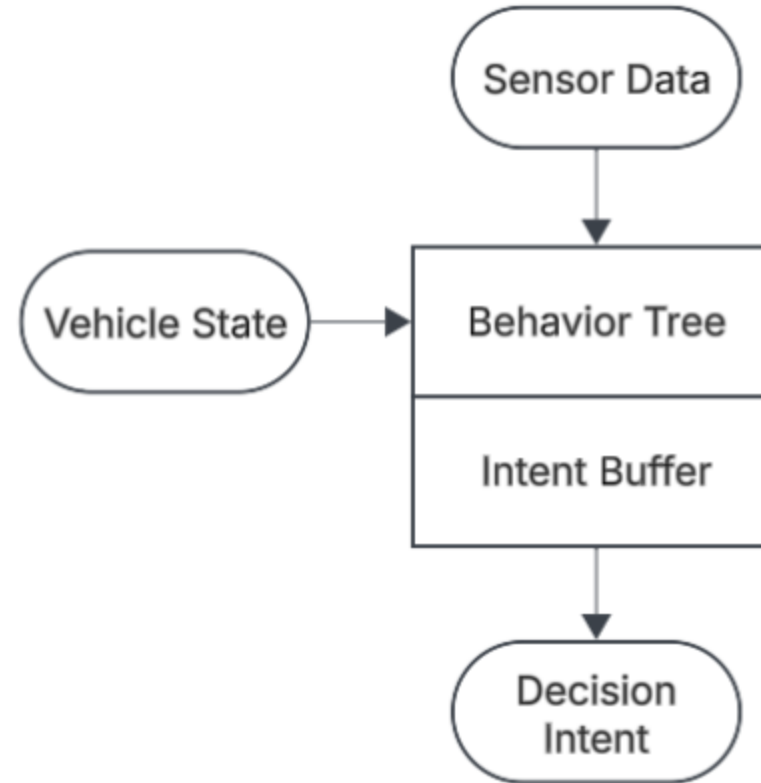
Control Layer

- Converting intent from decision layer to PX4 compatible flight commands.
- Interface between companion computer and flight controller.
- Safety directives take precedent over decision layer intent.
- Components:
 - Intent Validator
 - Offboard Interface

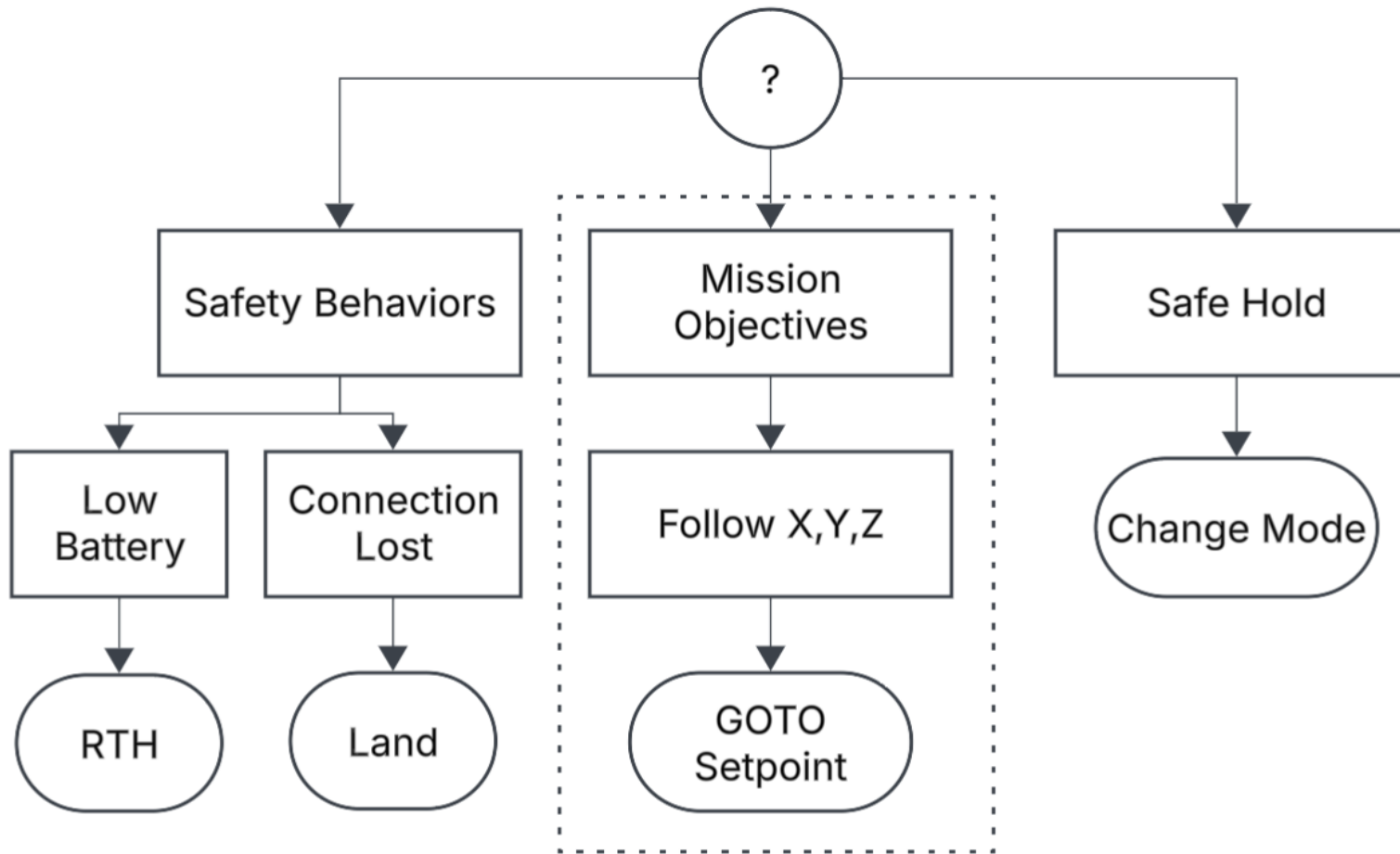


Decision Layer

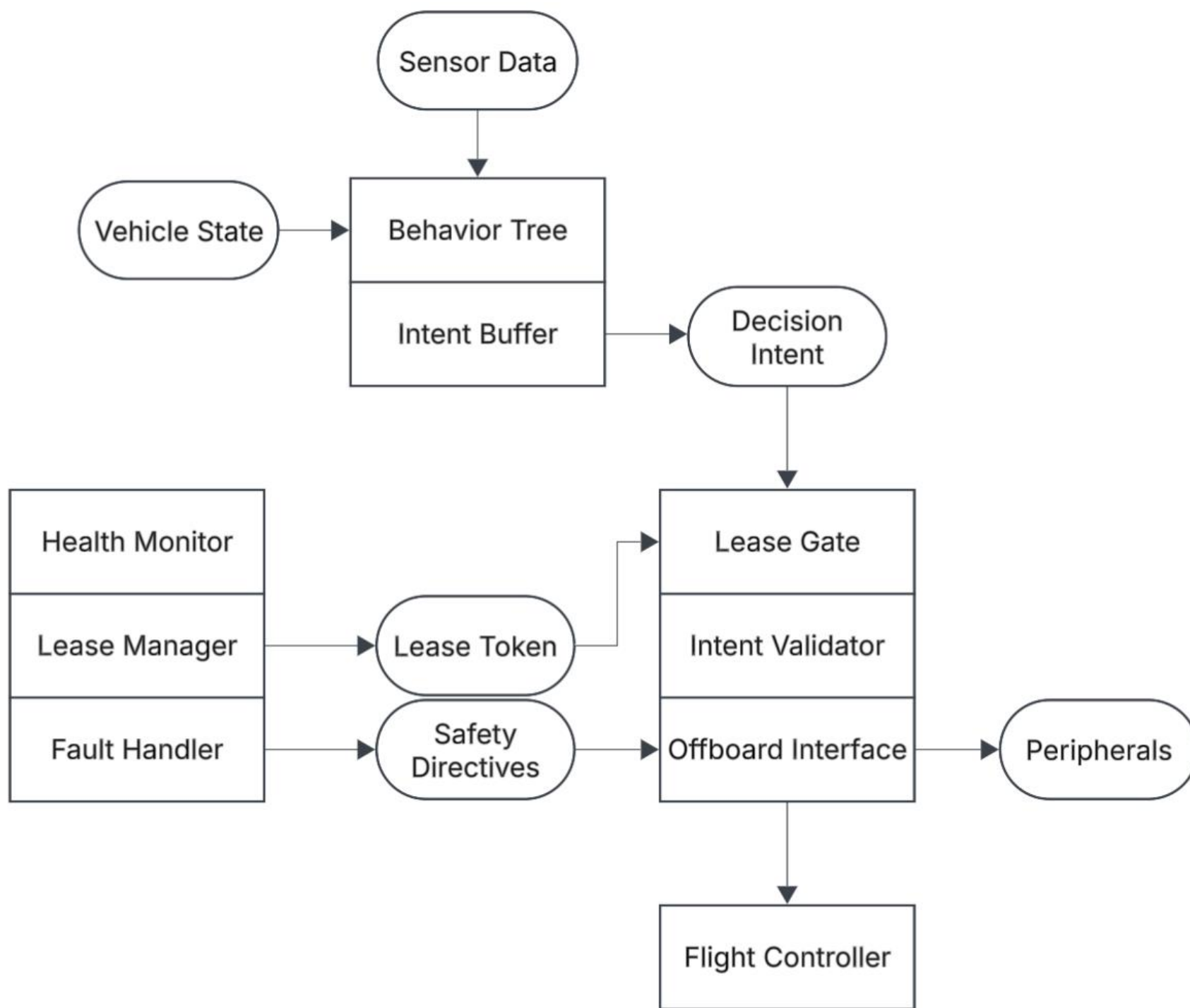
- What should the vehicle do?
- Makes decisions based on:
 - Mission objectives
 - Sensor data
 - Environmental context
 - Safety context
- Behavior Tree:
 1. Safety Behavior
 2. Mission objectives



Decision Layer – Behavior Tree



Use-case defined behavior



Developer Workflow

Cosys-AirSim

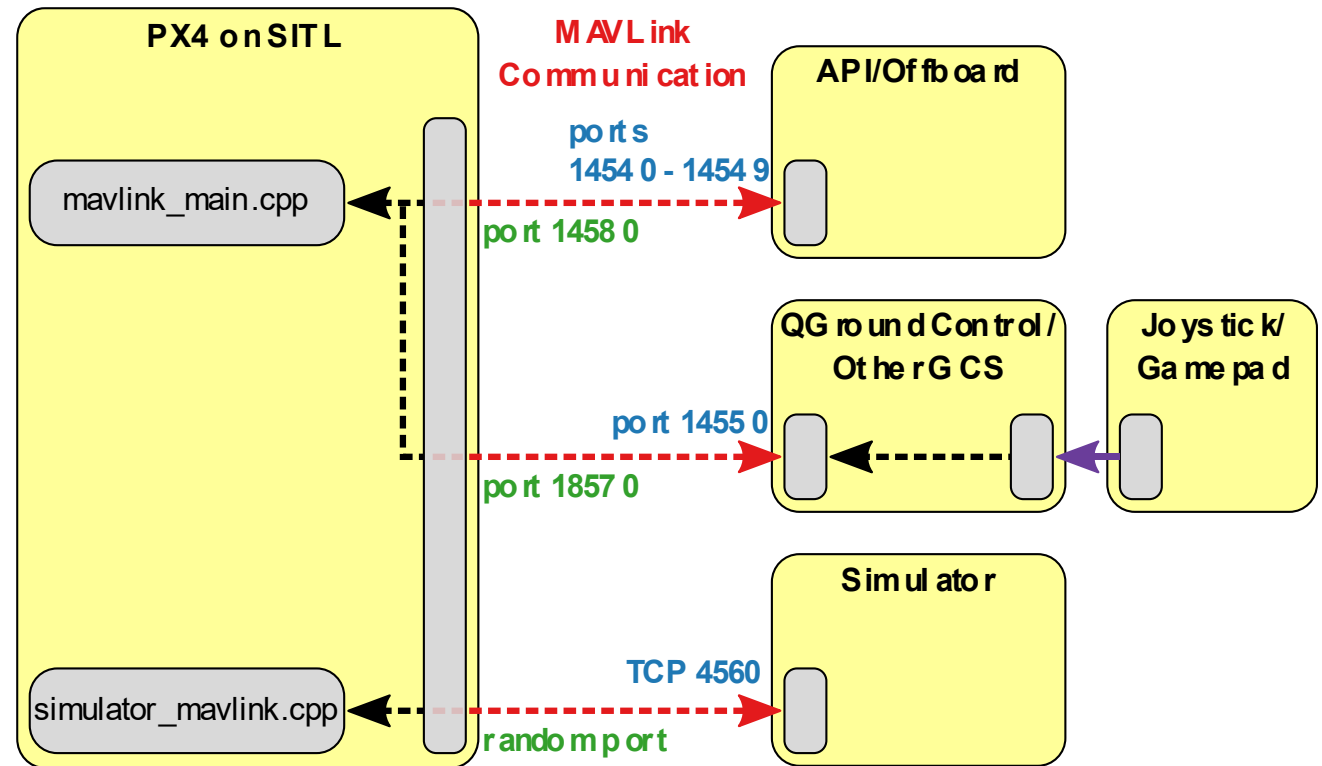
- Physics simulator for drones built on Unreal Engine. [8]
- Simulates full sensor suite:
 - Cameras
 - LiDAR
 - IMU
 - GPS
- Produces realistic data for PX4 SITL and companion computer.



Cosys-AirSim in UE5

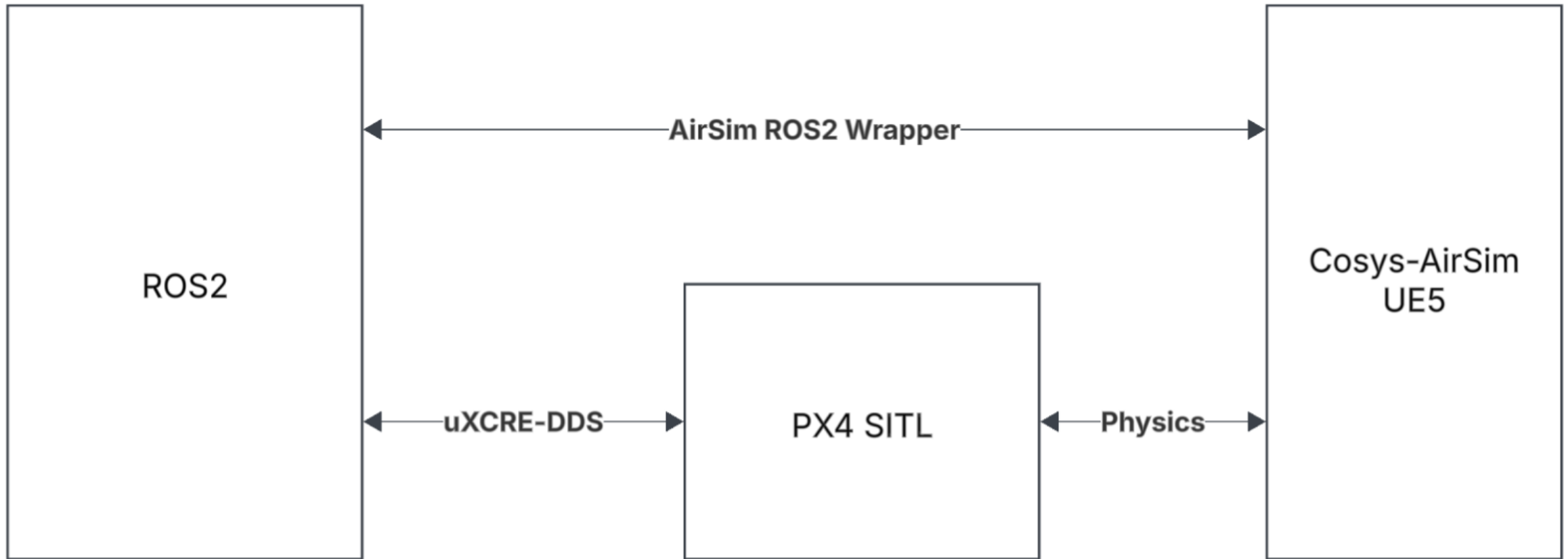
PX4 Software-In-The-Loop (SITL)

- Runs PX4 firmware on development computer. [9]
- Paired with a physics simulator like Cosys-AirSim for full vehicle simulation.
- Exposes same interface as a real PX4 for future deployment.



PX4 SITL Interfaces

Simulation Environment



ROS2 QoS

- Quality of Service policies define how messages between nodes are transmitted [2]
- Reliable QoS:
 - Lease State
 - Supervisor Directives
 - Fault Notifications
- Best-Effort QoS:
 - Sensor data topics
 - Vehicle state telemetry
 - Offboard handshake

ROS2 Lifecycle Nodes

- Lifecycles nodes are described in 4 states:
 - Unconfigured
 - Inactive
 - Active
 - Finalized
- Defines behavior when a node crashes or fails to transition state.
- Failure is detectable and not silent.

ROS2 Launch File

- Configure script that defines:
 - Node startup order.
 - What happens when a node crashes.
 - Pass config values to nodes.
- Node launch order:
 - Supervisor Layer Nodes
 - Control Layer Nodes
 - Decision Layer Nodes
- Ensures important infrastructure is up before execution can begin.

Limitations of the Architecture

- Safety guarantees are only as strong as the developer's implementation.
- PX4 is assumed reliable.
- Validated in simulation only.
- Resource overhead from supervisor processing.
- Scoped to single vehicle systems.

Future Work

- Hardware validation on physical PX4 platform.
- Provide additional references to implementation-level decisions.
 - Safety preconditions
 - Safe envelope
- Extend to multi-vehicle systems.

Challenges

- Over-scoped project
 - Research
 - Design
 - Physics Simulator
 - Full Implementation
- Limited Online Resources
 - No officially defined “Best Practices”
 - Initial goal of this project is to extend open-source reference

What did I enjoy?

- Robotics Research
- Learning across multiple domains
- Design

Questions and Discussion

Thank you!

References

- [1] “PX4 User Guide”, *docs.px4.io*. <https://docs.px4.io/main/en/>
- [2] “ROS 2 Documentation – ROS 2 Documentation: Humble documentation,” *docs.ros.org*. <https://docs.ros.org/en/humble/index.html>
- [3] "Aerostack2 Documentation," *aerostack2.github.io*. <https://aerostack2.github.io/>
- [4] "Nav2 Documentation," *nav2.org*. <https://nav2.org/>
- [5] "CTU MRS UAV System," *ctu-mrs.github.io*. <https://ctu-mrs.github.io/>
- [6] "MAVSDK FAQ," *mavsdk.mavlink.io*. <https://mavsdk.mavlink.io/v2.0/en/faq.html>
- [7] G. Dromey, "Behavior Trees: From Systems Engineering to Software Engineering," in *2010 8th IEEE International Conference on Software Engineering and Formal Methods*, Pisa, Italy, Sep. 2010, pp. 13–18. <https://ieeexplore.ieee.org/document/5637403/>
- [8] "Cosys-AirSim Documentation," *cosys-lab.github.io*. <https://cosys-lab.github.io/Cosys-AirSim/>
- [9] "PX4 Software-in-the-Loop (SITL) Simulation," *docs.px4.io*. <https://docs.px4.io/main/en/simulation/>